

Avaliação: Projeto - Unidade II

Instruções

Neste documento, você encontrará dois problemas distintos que devem ser resolvidos utilizando estruturas de dados específicas. Você deve escolher **um** dos problemas para solucionar.

- As soluções devem ser implementadas na linguagem C++.
- Para cada problema, além de desenvolver o algoritmo de solução, você deverá implementar as estruturas de dados necessárias para o correto funcionamento do programa. Isso inclui a implementação da pilha para o problema de verificação de parênteses balanceados e a implementação das filas para o problema de gestão de pacientes.

Problema 1: Verificação de Parênteses Balanceados

Descrição do Problema

Neste projeto, você deve desenvolver um programa em C++ que verifique se uma expressão contém parênteses balanceados. Parênteses balanceados significam que cada parêntese de abertura tem um correspondente parêntese de fechamento e que os pares de parênteses estão corretamente aninhados.

Por exemplo:

- A expressão $(a+b)$ é balanceada.
- A expressão $((a+b)$ não é balanceada, pois há um parêntese de abertura extra.
- A expressão $(a+b)*(c-d)$ é balanceada.
- A expressão $(a+b)*(c-d))$ não é balanceada, pois há um parêntese de fechamento extra.

Além de verificar os parênteses '()', o programa deve ser capaz de verificar outros tipos de delimitadores: colchetes '[]' e chaves '{}'. A verificação deve garantir que todos os delimitadores estão balanceados e corretamente aninhados.

Especificação

O programa deve:

1. Receber uma expressão como entrada, que pode conter qualquer combinação de parênteses, colchetes e chaves, além de outros caracteres como letras, números e operadores.
2. Utilizar uma estrutura de dados **Pilha** para realizar a verificação de balanceamento dos delimitadores.
3. Retornar uma mensagem indicando se a expressão está **balanceada** ou **não balanceada**.

Estratégia de Solução

1. Percorra a expressão caractere por caractere.
2. Quando encontrar um parêntese de abertura '(', um colchete de abertura '[' ou uma chave de abertura '{', empilhe o caractere na pilha.
3. Quando encontrar um parêntese de fechamento ')', um colchete de fechamento ']' ou uma chave de fechamento '}', verifique o topo da pilha:
 - (a) Se o topo da pilha contiver o correspondente caractere de abertura, desempilhe-o e continue.
 - (b) Se o topo da pilha não corresponder ao caractere de fechamento, ou se a pilha estiver vazia, a expressão não está balanceada.
4. Após percorrer toda a expressão, verifique o estado da pilha:
 - (a) Se a pilha estiver vazia, todos os delimitadores foram balanceados, então a expressão está balanceada.
 - (b) Se ainda houver elementos na pilha, isso significa que há delimitadores de abertura sem correspondentes de fechamento, portanto, a expressão não está balanceada.

Entrada

A entrada será uma string que representa a expressão a ser verificada. A expressão pode conter letras, números, operadores matemáticos, e os delimitadores '()', '[]', '{'}.

Saída

A saída será uma mensagem indicando se a expressão está balanceada ou não.

Exemplo de saída: "A expressão está balanceada" ou "A expressão não está balanceada".

Exemplos

- Entrada: "(a+b)*(c-d)"
- Saída: "A expressão está balanceada"
- Entrada: "(a+b))"
- Saída: "A expressão não está balanceada"
- Entrada: "[a+b]*(c-d)"
- Saída: "A expressão está balanceada"
- Entrada: "[(a+b)]"
- Saída: "A expressão não está balanceada"

Problema 2: Gestão de Pacientes em uma Clínica

Descrição do Problema

Uma clínica médica possui um sistema de atendimento aos pacientes onde as consultas são realizadas por ordem de chegada. No entanto, pacientes com 60 anos ou mais têm prioridade no atendimento. Para gerenciar a ordem de atendimento, a clínica utiliza duas filas de espera: uma para pacientes normais (abaixo de 60 anos) e outra para pacientes prioritários (60 anos ou mais).

Você deve implementar um programa em C++ que simule o sistema de fila da clínica, permitindo adicionar pacientes às filas, atender os pacientes de acordo com as prioridades, e verificar o estado atual das filas (quem está esperando).

Especificação

O programa deve permitir as seguintes operações:

1. **Adicionar paciente à fila:** O nome e a idade do paciente são fornecidos. Dependendo da idade, o paciente é adicionado à fila normal ou à fila prioritária.
2. **Atender paciente:** O paciente na fila prioritária é atendido primeiro. Se a fila prioritária estiver vazia, o próximo paciente na fila normal é atendido.
3. **Verificar filas:** O programa deve exibir todos os pacientes que ainda estão na fila prioritária e na fila normal, na ordem em que serão atendidos.

Estratégia de Solução

1. Utilize duas estruturas de dados **Fila**, uma para pacientes normais e outra para pacientes prioritários.
2. Cada vez que um paciente é adicionado, sua idade é verificada:
 - Se o paciente tiver 60 anos ou mais, ele é adicionado à fila prioritária.
 - Se o paciente tiver menos de 60 anos, ele é adicionado à fila normal.
3. Quando um paciente é atendido, primeiro verifique se há pacientes na fila prioritária. Se houver, atenda o paciente da fila prioritária. Caso contrário, atenda o paciente da fila normal.
4. A qualquer momento, as filas podem ser verificadas para mostrar a ordem dos pacientes.

Entrada

A entrada será composta por uma série de comandos:

- **ADICIONAR <nome_do_paciente> <idade>:** Adiciona o paciente à fila correspondente com base na idade.
- **ATENDER:** Remove o paciente na fila prioritária, se houver, ou da fila normal e marca-o como atendido.
- **VERIFICAR:** Exibe os pacientes que ainda estão na fila prioritária e na fila normal.

Saída

A saída do programa deve ser:

- Mensagens confirmando a adição de pacientes às filas.
- Mensagens indicando o paciente que foi atendido e a qual fila ele pertencia.
- As listas atuais de pacientes nas filas quando solicitado.

Exemplos

- Entrada:

```
ADICIONAR Maria 65
ADICIONAR João 30
VERIFICAR
ATENDER
VERIFICAR
```

- Saída:

Paciente Maria (Prioritário) adicionado à fila.

Paciente João (Normal) adicionado à fila.

Fila Prioritária: Maria

Fila Normal: João

Paciente Maria (Prioritário) foi atendido.

Fila Prioritária:

Fila Normal: João